

Calcolatori Elettronici

6 settembre 2023

Teoria

Esercizio 1 (12 punti): Progettare un automa a stati finiti che accetti in ingresso una sequenza di caratteri appartenenti all'alfabeto di input $I = \{A, B\}$ e produca l'output "1" ogni volta che rileva la sequenza "ABABA". Fornire il diagramma di stato, la tabella di transizione e una breve spiegazione del funzionamento dell'automa. Non è necessario sintetizzare il circuito.

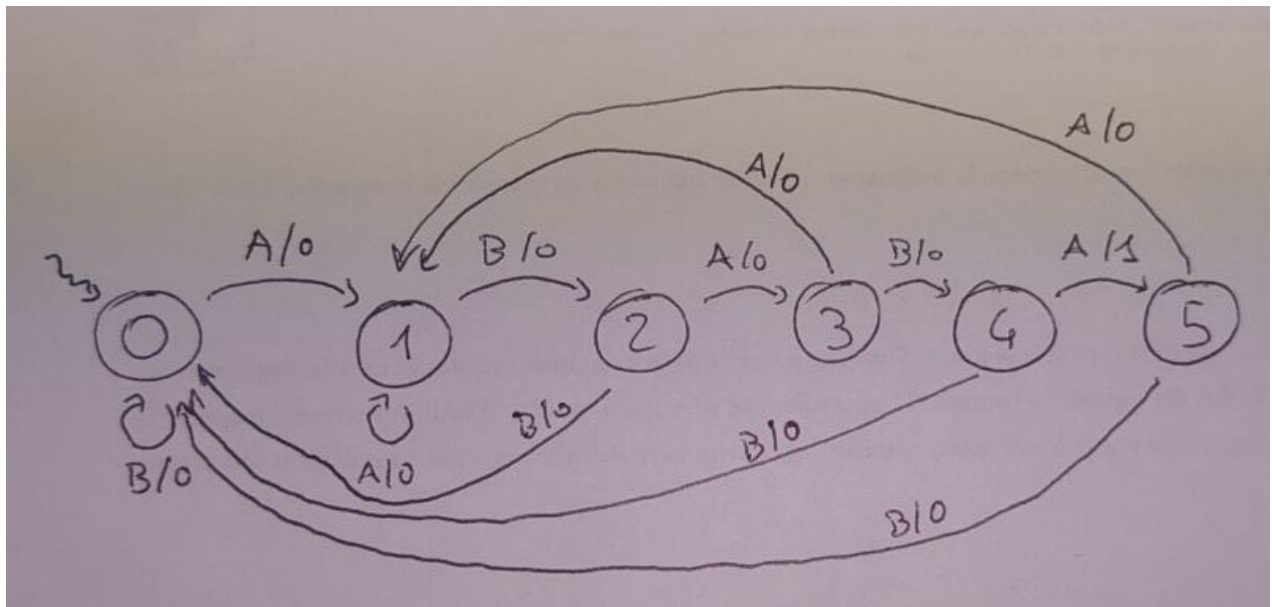
Esercizio 2 (12 punti): Convertire il numero decimale 9.5 in formato in virgola mobile a 8-bit con 1 bit per il segno, 3 bit per l'esponente, e 4 bit per la mantissa. Dettagliare ogni passaggio della conversione e fornire il numero in formato in virgola mobile risultante.

Esercizio 3 (6 punti): Illustrare il concetto di *busy waiting* e come esso differisce dall'uso delle interruzioni. Discutere le implicazioni di ciascun approccio in termini di efficienza del sistema.

Soluzioni

Esercizio 1

Si propone la soluzione per un automa a stati finiti che riconosce *sequenze non sovrapposte*.



Per la codifica degli stati, si utilizza la rappresentazione in binario dei numeri. Per l'input, si utilizza come rappresentazione $A = 0$ e $B = 1$. La tabella di transizione sarà quindi:

y_0	y_1	y_2	x	y'_0	y'_1	y'_2
0	0	0	0	0	0	1
0	0	0	1	0	0	0
0	0	1	0	0	0	1
0	0	1	1	0	1	0
0	1	0	0	0	1	1
0	1	0	1	0	0	1
0	1	1	0	0	0	1
0	1	1	1	1	0	0
1	0	0	0	1	0	1
1	0	0	1	0	0	0
1	0	1	0	0	0	1
0	1	0	1	0	0	0

Esercizio 2

Nell'esercizio dato, il numero da convertire è 9.5. Seguiamo i seguenti passaggi:

- Rappresentazione in Binario:** Innanzitutto, convertiamo il numero 9.5 in binario. Il numero intero 9 in binario è `1001`, e la parte decimale 0.5 in binario è `0.1`. Quindi, il numero completo è `1001.1` in binario.
- Normalizzazione:** Per normalizzare, spostiamo la virgola in modo che ci sia un solo bit diverso da zero a sinistra della virgola. Questo ci dà $1.0011 \cdot 2^3$.
- Codifica del Segno:** Il numero è positivo, quindi il bit del segno è `0`.
- Codifica dell'Esponente:** Con 3 bit per l'esponente e un bias di 3, l'esponente codificato è $(3 + 3 = 6)$, che è `110` in binario.
- Codifica della Mantissa:** Dopo la normalizzazione, la mantissa è `1.0011`. Poiché la parte a sinistra della virgola è sempre 1 per i numeri normalizzati, possiamo ometterla e considerare solo la parte decimale `0011`.
- Rappresentazione finale:** Mettendo insieme tutte le parti, otteniamo:
 - Bit del segno: (0)
 - Esponente: (110)
 - Mantissa: (0011)

In definitiva, la rappresentazione in virgola mobile a 8 bit del numero 9.5 è 01100011.

Progetto

Un processore z64 controlla la centralina di un'automobile autonoma per l'arresto di emergenza del veicolo. Il sistema è composto da una periferica TELECAMERA e RADAR. La periferica TELECAMERA viene interrogata in maniera sincrona dal processore, per acquisire delle immagini di dimensione 800x600 pixel in bianco e nero—ciascun pixel ha dimensione un byte. L'acquisizione delle immagini viene realizzata utilizzando il DMAC di sistema.

Le immagini acquisite sono delle *mappe di profondità*: un pixel nero (codificato come 0x00) rappresenta un punto in cui non ci sono oggetti nelle vicinanze, mentre un pixel bianco (codificato come 0xFF) rappresenta un punto in cui è presente un oggetto a distanza inferiore a 20 metri. Il processore utilizza queste immagini per pilotare la periferica sincrona ARRESTO che effettua la frenata di emergenza, qualora il numero di pixel bianchi sia maggiore di 1/4 dei pixel totali.

La periferica RADAR, invece, invia una richiesta di interruzione al processore z64 ogni volta che viene rilevato un oggetto in prossimità. Nel caso in cui il sistema rilevi un tale oggetto, il processore attiva la periferica ARRESTO per effettuare la frenata di emergenza.

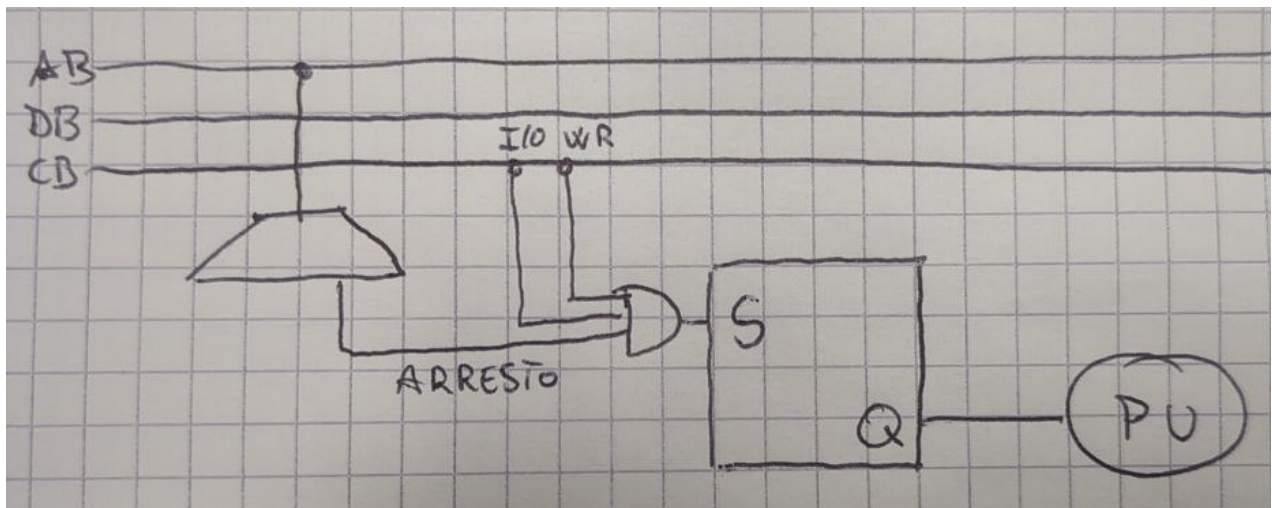
Realizzare:

- Le interfacce delle periferiche TELECAMERA, RADAR e ARRESTO;
- Tutto il software necessario al funzionamento del sistema, comprensivo di eventuali driver.

Soluzione

Si propone una possibile soluzione. Per errore, è stato indicato nelle interfacce PU al posto di CU.

La periferica TELECAMERA è una periferica sincrona, pertanto prima di poter effettuare l'acquisizione dell'immagine è necessario avviarla, per chiedere di effettuare uno "scatto". Quando la periferica completa questa operazione, l'immagine è disponibile nella sua memoria interna che è interfacciata al DMAC di sistema (pertanto, non è necessario realizzare l'interfaccia con il processore). Lo schema è quindi il seguente. Si noti che la linea di indirizzo utilizzata per indirizzare il flip flop STATUS dovrà essere differente dall'indirizzo utilizzato dal DMAC per interagire con il dispositivo.



Il codice che guida il sistema può essere il seguente.

```

1  .org 0x800
2  .data
3      .equ TELECAMERA, 0x00
4      .equ TELECAMERA_BW, 0x01
5      .equ RADAR, 0x02
6      .equ ARRESTO, 0x03
7      buffer: .fill 480000    # Allocazione di 800*600 byte per l'immagine
8
9  .text
10 main:
11     .poll_loop:
12         call request_image
13         call process_and_check_image
14         jmp .poll_loop
15
16 request_image:
17     outb %al, $TELECAMERA
18     .bw:
19     inb $TELECAMERA, %al
20     btb $0, %al
21     jnc .bw
22     movq $buffer, %rdi
23     movq $480000, %rcx
24     movw $TELECAMERA_BW, %dx
25     cld
26     insb
27     ret
28
29 process_and_check_image:
30     xorl %eax, %eax # Inizializzazione del conteggio dei pixel a 0
31
32     # Elaborazione ciascun pixel dell'immagine
33     movl $480000, %ecx # 800*600
34     leal buffer, %ebx
35
36     .process_loop:
37     movb (%ebx), %dl
38     cmpb $0xFF, %dl
39     jne .no_white
40     addl $1, %eax
41     .no_white:

```

```

42     addl $1, %ebx
43     loop .process_loop
44
45     # Verifica se è necessario attivare ARRESTO
46     cmpl $120000, %eax # 1/4 dei pixel totali
47     jl .no_brake
48     call activate_brake
49 .no_brake:
50     ret
51
52 activate_brake:
53     outb %al, $ARRESTO
54     ret
55
56 # Gestore delle richieste di interruzione per il RADAR
57 .driver 0:
58     call activate_brake
59     iret

```

Prova di laboratorio

Siete stati assunti come sviluppatori software per una piccola startup che si occupa di e-commerce. La vostra prima missione è di scrivere un programma in C che aiuti l'azienda a gestire il suo magazzino. Il programma deve permettere agli operatori di inserire nuovi prodotti, eliminare prodotti esistenti e visualizzare l'elenco dei prodotti disponibili. Ogni prodotto è caratterizzato da un nome e un prezzo.

Il programma è interattivo: all'avvio, viene presentato un menu testuale del tipo:

```

1  1. Inserire prodotto
2  2. Eliminare prodotto
3  3. Visualizza prodotti
4  4. Uscita

```

L'utente, inserendo uno dei numeri associati alle voci di menu, può utilizzare in maniera interattiva la funzionalità richiesta.

Non c'è limite al numero di prodotti che possono essere inseriti. Per semplicità, l'inventario è mantenuto unicamente in memoria (all'uscita del programma, tutti i dati inseriti vengono persi).

Soluzione

Una possibile soluzione è la seguente. L'unico aspetto rilevante è l'utilizzo della `scanf` per acquisire valori numerici. Si è scelto di presentare tale soluzione a valle di numerose richieste di studenti che chiedevano in che modo si potesse utilizzare comunque la `scanf` per realizzare operazioni di input. Il controllo degli errori esplicito che viene effettuato in questo caso ed il conseguente svuotamento dello `stdin` permette di rendere il codice sufficientemente robusto. Tuttavia, come discusso a lezione, questo utilizzo della `scanf` non permette di accorgersi di eventuali errori di conversione. Ad esempio, `1a` è un input considerato valido (ed effettivamente l'input viene interpretato come `1`). L'uso di `strtoul`, `strtoul` e simili è sempre incoraggiato.

Si noti che per l'input testuale si è comunque utilizzato `fgets`.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  struct product {
6      char *name;
7      float price;
8  };
9
10 struct product *products;
11 int num_products = 0;
12 int capacity = 1;
13
14 int get_int_input(const char *prompt) {
15     int input;
16     printf("%s", prompt);
17     while (scanf("%d", &input) != 1) {
18         printf("Invalid input.\n");
19         while (getchar() != '\n');
20         printf("%s", prompt);
21     }
22     while (getchar() != '\n');
23     return input;
24 }
25
26 void add_product(char *name, float price) {
27     if (num_products == capacity) {
28         struct product *temp = realloc(products, capacity * 2 * sizeof(struct product));
29         if (temp == NULL) {
30             printf("Memory allocation failed.\n");
31             return;
32         }
33         products = temp;
34         capacity *= 2;
35     }
36
37     products[num_products].name = strdup(name);
38     if (products[num_products].name == NULL) {
39         printf("Memory allocation failed.\n");
40         return;
41     }
42
43     products[num_products].price = price;
44     num_products++;
45 }
46
47 void delete_product(int index) {
48     if (index < 0 || index ≥ num_products) {
49         printf("Invalid index.\n");
50         return;
51     }
52
53     free(products[index].name);
54
55     for (int i = index; i < num_products - 1; i++) {
56         products[i] = products[i + 1];
57     }
58
59     num_products--;
60 }
61

```

```

62 void show_products(void) {
63     printf("Product List:\n");
64     for (int i = 0; i < num_products; i++) {
65         printf("%d. %s: %.2f\n", i, products[i].name, products[i].price);
66     }
67 }
68
69 void free_memory(void) {
70     for (int i = 0; i < num_products; i++) {
71         free(products[i].name);
72     }
73     free(products);
74 }
75
76 int main(void) {
77     products = malloc(capacity * sizeof(struct product));
78     if (products == NULL) {
79         printf("Memory allocation failed.\n");
80         return 1;
81     }
82
83     char name[50];
84     float price;
85     int choice, index;
86
87     while (1) {
88         choice = get_int_input("1. Add Product\n2. Delete Product\n3. Show Products\n4. Exit\n");
89         switch (choice) {
90             case 1:
91                 printf("Enter product name: ");
92                 if (fgets(name, sizeof(name), stdin) == NULL) {
93                     printf("Input error.\n");
94                     continue;
95                 }
96                 name[strcspn(name, "\n")] = 0;
97                 printf("Enter product price: ");
98                 if (scanf("%f", &price) != 1) {
99                     printf("Invalid input.\n");
100                     while (getchar() != '\n');
101                     continue;
102                 }
103                 while (getchar() != '\n');
104                 add_product(name, price);
105                 break;
106
107             case 2:
108                 index = get_int_input("Enter the index of the product to delete: ");
109                 delete_product(index);
110                 break;
111
112             case 3:
113                 show_products();
114                 break;
115
116             case 4:
117                 free_memory();
118                 return 0;
119
120             default:
121                 printf("Invalid choice.\n");
122         }
123     }

```